COMMUNITY

PSP EBOOT
BUILDING FOR OFW

A practical guide to native User Mode applications, the legacy signing envelope, and independent verification.

INPUT SAFETY

4 MiB

4,194,304 bytes / 0x400000

VALIDATION

Paranoid

Cross-Validation

Worked example: SampleApp
Built, signed, decrypted, cross-checked, and tested with damaged controls on a computer. Physical PSP / OFW testing remains a separate, uncompleted gate.

Target firmware: OFW 6.60 / 6.61

English edition 1.0 | 11 October 2026

How to use this guide

A reproducible development path, with explicit limits on every compatibility claim.

Start with a native PSP program that uses public User Mode interfaces. Package its PRX in a PBP, apply the documented legacy signing format, and verify the transformation independently. Then test the exact output on physical official firmware.

Pages	What you will find
3	PBP, PRX and ELF: the file formats and their roles
4-7	SampleApp source, Makefile, build workflow and import audit
8-10	Signing mechanism, the 4 MiB input buffer and a bounded preflight
11-13	Paranoid Cross-Validation: structure, permissions and cryptographic checks
14-16	Complete independent Python verifier, in three consecutive parts
17-18	Damaged controls and the measured SampleApp audit record
19-20	Memory planning, physical PSP tests and the installation layout
21-22	Troubleshooting and a community-ready release package
23	Primary sources, pinned revisions and provenance

Evidence status

The neutral example was compiled and signed in this guide's preparation. Independent offline verification, size-boundary controls and five damaged-artifact controls passed. No physical PSP, OFW launch or emulator result is claimed for SampleApp.

Prerequisites: a Linux or WSL environment, a working PSPDEV toolchain, the `ebootsign` executable, Python 3, and the Python `cryptography` package for the independent verifier. PSPDEV's primary documentation is the installation starting point [S1].

The worked path is specifically **pspdev/ebootsigner** at the revision listed on page 23. A similarly named utility or another signer can use a different format, limit or security model. Re-check its implementation before applying these numbers.

Embedded examples: main.c, Makefile, preflight_pbp.py, cross_validate.py and LICENSE.txt are attached inside this PDF. A desktop reader with attachment support can save their exact bytes. The printed examples remain usable without attachment support.

What an EBOOT contains

PBP is a container. PRX is the executable module carried inside it.

EB00T.PBP is the conventional filename used by the PSP's XMB application launcher. Its name alone says nothing about the executable's authenticity, permissions or firmware compatibility.

Section	Purpose	SampleApp input
PARAM.SFO	Application title and other launch metadata	316 bytes
ICON0.PNG	Static XMB icon	Empty
ICON1.PMF	Animated XMB preview	Empty
PIC0.PNG	Small preview graphic	Empty
PIC1.PNG	Background graphic	Empty
SND0.AT3	XMB preview sound	Empty
DATA.PSP	Executable payload: raw PRX before signing	158,450 bytes
DATA.PSAR	Optional additional payload	Empty

The PBP header is 40 bytes: a 4-byte magic, a version word and eight little-endian section offsets. The next offset marks the end of a section; file length marks the end of DATA.PSAR. Equal offsets are legal when a section is empty. The measured sample uses version `0x00010000`.

The unsigned DATA.PSP starts with `7F 45 4C 46`: ELF magic. This signing path expects an ELF32, little-endian MIPS **relocatable PSP PRX**, with `e_type = 0xFFA0`. A desktop ELF, plain text or arbitrary renamed file does not satisfy that requirement.

> **Three related outputs**
>
> SampleApp.elf is the linker's ELF output; SampleApp.prx is the PSP module produced for packaging; EBOOT.PBP is the outer launch container. In this example, unsigned DATA.PSP is exactly the generated SampleApp.prx. The signed payload instead begins with a ~PSP wrapper.

Format and packing details are grounded in the pinned packer/unpacker and relocation code [S4]. The section sizes in this guide are measured from the example, not generic size limits.

A minimal User Mode program

Save this complete example as main.c. It draws text and services the HOME exit callback.

```
/* SPDX-License-Identifier: MIT */
#include <pspkernel.h>
#include <pspdebug.h>

PSP_MODULE_INFO("SampleApp", 0, 1, 0);
PSP_MAIN_THREAD_ATTR(PSP_THREAD_ATTR_USER);
PSP_MAIN_THREAD_STACK_SIZE_KB(32);
PSP_HEAP_SIZE_KB(2048);

static int quit;

static int on_exit(int a, int b, void *p)
{
    (void)a; (void)b; (void)p;
    __atomic_store_n(&quit, 1, __ATOMIC_RELAXED);
    return 0;
}

int main(void)
{
    SceUID cb = sceKernelCreateCallback("SampleExit", on_exit, NULL);
    if (cb < 0 || sceKernelRegisterExitCallback(cb) < 0) {
        sceKernelExitGame();
        return 1;
    }
    pspDebugScreenInit();
    pspDebugScreenPrintf("SampleApp - User Mode\nPress HOME to exit.\n");
    while (!__atomic_load_n(&quit, __ATOMIC_RELAXED))
        sceKernelDelayThreadCB(20000);
    sceKernelExitGame();
    return 0;
}
```

Module attribute 0: the source declares a user module. **USER main-thread attribute:** the thread is explicitly created for User Mode. The example requests a 32 KiB main-thread stack and a 2 MiB heap; those values are deliberately separate from the signer's 4 MiB host input buffer [S2].

The exit callback sets a flag. The main loop uses `sceKernelDelayThreadCB`, so callbacks are serviced instead of leaving the HOME action waiting forever. If callback creation or registration fails, the program exits. The atomic flag avoids relying on an unsynchronized shared variable.

Including `pspkernel.h` makes SDK declarations available; it does not itself grant kernel privileges. Public functions with names beginning `sceKernel` can be callable from a user application. Audit the actual imported library, NID and stub, rather than the spelling of a function name [S2, S3].

Build settings that matter

Save this Makefile beside main.c. Recipe indentation belongs to the included SDK rules.

```
TARGET = SampleApp
OBJS = main.o
CFLAGS = -O2 -G0 -Wall -Wextra -Werror -ffunction-sections -fdata-sections
LDFLAGS = -Wl,--gc-sections,-Map,build/SampleApp.map
LIBS =
BUILD_PRX = 1
PSP_FW_VERSION = 660
PSP_LARGE_MEMORY = 0
PSP_EBOOT_TITLE = SampleApp
SFOFLAGS += -s PSP_SYSTEM_VER=6.60
EXTRA_TARGETS = EBOOT.PBP
PSPSDK := $(shell psp-config --pspsdk-path)
include $(PSPSDK)/lib/build.mak
```

Setting	Why it is present
BUILD_PRX = 1	Generate a PRX for this signer, rather than relying on a plain ELF.
PSP_FW_VERSION = 660	Select the SDK firmware interface target; this is not a hardware test.
PSP_LARGE_MEMORY = 0	Do not request the SDK's large-memory SFO option.
PSP_SYSTEM_VER=6.60	Set minimum firmware metadata explicitly in PARAM.SFO.
-Wall -Wextra -Werror	Treat compiler warnings as failures for this small example.
build/SampleApp.map	Preserve the linker map for dependency and import investigation.

Create `build/` before running make, because the map file is written there. `PSP_FW_VERSION` and `PSP_SYSTEM_VER` address different layers: a compile/link configuration does not automatically prove the SFO metadata is correct [S5].

After changing SFO flags, regenerate PARAM.SFO and EBOOT.PBP in your disposable build workspace. An existing SFO can otherwise remain up to date according to make's timestamp rules. The measured example contains TITLE=SampleApp, CATEGORY=MG, BOOTABLE=1, MEMSIZE=0 and PSP_SYSTEM_VER=6.60.

Build, preflight, then sign

Keep the original unsigned file and run each legacy signing job in its own working directory.

Install PSPDEV using its primary instructions [S1]. In WSL, execute the build inside the Linux environment; use a normal Linux project directory for the toolchain and intermediate files. The following assumes the toolchain is installed at `$HOME/pspdev` and ebootsign is available in its bin directory.

```
set -eu
export PSPDEV="$HOME/pspdev"
export PATH="$PSPDEV/bin:$PATH"
psp-gcc --version
psp-config --pspsdk-path
command -v ebootsign

mkdir -p build
make -j2
python3 preflight_pbp.py EBOOT.PBP
psp-readelf -h -S -l SampleApp.prx

mkdir -p build/sign
(
    cd build/sign
    ebootsign ../../EBOOT.PBP ../EBOOT-signed.PBP
)
sha256sum EBOOT.PBP build/EBOOT-signed.PBP
```

The preflight code is on page 10. The shell's `set -e` stops this sequence when that gate fails. Before signing, review the PRX identity, section/program ranges, module attributes and imports as described on pages 7 and 12. Only use this legacy tool on your own audited build input.

> **Isolation is functional, not cosmetic**
>
> The pinned tool uses a fixed temporary directory named tmp_sgn and fixed intermediate filenames. It also removes an existing output before creating a replacement. Concurrent signing jobs must use different working directories and different output paths [S4].

A zero process exit code is one observation, not the final verdict. The output must also pass the independent verifier, section comparison and device test plan. Do not overwrite the known-good distributable until the new candidate has passed its required gates.

For a repeated job, create another clean job directory. Audit and sign again after changes to executable code or linked dependencies. Changed PBP metadata or graphics also need fresh structural and device checks.

Audit what the module imports

User Mode compatibility depends on the actual imported interfaces, not only a source flag.

The measured SampleApp PRX contains **11 import libraries and 49 function NIDs**. SDK startup and C runtime code can introduce dependencies that are absent from main.c. For example, this build includes networking stubs through the runtime even though the application has no networking feature.

Imported library	Count	Imported library	Count
sceDisplay	2	Kernel_Library	2
sceGe_user	1	ModuleMgrForUser	1
sceUtility	1	StdioForUser	3
sceNetInet	4	SysMemUserForUser	4
IoFileMgrForUser	9	ThreadManForUser	20
LoadExecForUser	2	Total	49

```
psp-readelf -h -S -l SampleApp.prx
psp-readelf -x .lib.stub -x .rodata.sceNid SampleApp.prx
psp-nm -n SampleApp.elf
```

Read each `.lib.stub` descriptor and resolve its pointers against the PRX's section address ranges. Record the library name, version/attributes, function count and NID array. Match those NIDs to the SDK's user stub definitions and check the associated linker-map archive members.

A useful counterexample

Kernel_Library in this build imports NIDs 0xBEA46419 and 0x15B6446B, which the SDK user stubs identify as sceKernelCpuSuspendIntr and sceKernelCpuResumeIntr [S3]. The library name contains 'Kernel'; the relevant evidence is the user stub implementation and supported call path.

A user flag cannot make a CFW-only export available on OFW. Remove dependencies on private CFW interfaces, privileged modules or unsupported firmware exports. Library names alone are insufficient: confirm exact NIDs, import attributes, module availability and behavior on each target firmware.

What this signer really does

The term 'signed homebrew' is convenient; the specific implementation is more precise.

The pinned ebootsign implementation unpacks the PBP, processes the PRX relocations, constructs an encrypted KIRK-format body, and repacks the original container resources with a new DATA.PSP [S4]. The cryptographic processing happens in software on the computer.

Stage	Observed implementation
Unpack	Extract the PBP sections into temporary files.
Relocation adjustment	For SHT 0x700000A0 entries, clear a low-byte relocation type of 7 to 0.
Envelope preparation	Use fixed ~PSP and KIRK header templates and the bundled public implementation's constants.
Encryption	Encrypt the padded payload using the AES-CBC path implemented by the tool.
CMAC correction	Restore the template header and call kirk_forge to correct a body block for the expected CMAC.
Repack	Write the fixed wrapper plus encrypted body as DATA.PSP and rebuild the PBP.

This is a **specific legacy AES/CMAC envelope technique**. It is not a general Sony publisher certificate or a universal ECDSA signing capability. Calling it 'KIRK signing' should not imply that the application's main thread directly controls the console's KIRK hardware.

The ~PSP wrapper uses a fixed template. Its recorded sizes and outer fields describe that template, not a newly generated summary of every application. The final payload is therefore substantially larger than the small SampleApp PRX.

> **Format-specific instructions**
>
> This guide covers the executable named ebootsign from pspdev/ebootsigner. Utilities named ebootsigner, EasyPBPRX or similar should be assessed separately. Do not assume they share this buffer size, wrapper or verification recipe.

The software verifier later in this guide reconstructs this exact format. It verifies specified offline properties; it does not emulate the complete firmware loader or establish acceptance on every PSP model.

The 4 MiB input buffer

A concrete memory-safety limit in the inspected host signing implementation.

Exact value and scope

4 MiB = 4,194,304 bytes = 0x400000. This is the size of the signer's raw input PRX array, declared as elf[MAX_ORIGIN_FILE_SIZE]. It applies to unsigned DATA.PSP, not the entire PBP, the final encrypted output or the PSP application's runtime heap.

In the inspected code, MAX_ORIGIN_FILE_SIZE is defined as 4 * 1024 * 1024. The loading routine obtains the file length and passes that length to fread into the fixed array. It does **not** first reject a length greater than the array capacity [S4].

A later check compares the already-read input against `0x553410 - 0x110 = 0x553300`, or 5,583,616 bytes. That later threshold is larger than the 4 MiB input array and occurs too late to protect the read. It must not be used as the safe input limit.

Input DATA.PSP length	Bounded gate	Interpretation
4,194,303 bytes	Accept	One byte below the input-array limit.
4,194,304 bytes	Accept	Exactly fits; all other format checks still apply.
4,194,305 bytes	Reject	One byte beyond capacity; never pass this to the legacy signer.

Those boundary outcomes were exercised with synthetic fixtures using only the preflight gate. The fixtures are not complete executable PRXs, and accepting a size boundary does not establish that they are runnable. The oversized fixture was never sent to ebootsign.

If your real PRX is too large

Keep bulky assets outside the executable, remove unused linked code, and inspect the linker map. Choose a different reviewed signing implementation or repair this tool with a pre-read capacity check. Simply increasing one array leaves the fixed cryptographic layout and other unchecked structures unchanged.

Use MiB for binary units: 1 MiB is 1,048,576 bytes. A decimal MB is 1,000,000 bytes. The source's arithmetic, not an informal '4 MB' label, determines this boundary.

A bounded preflight gate

Save the complete script below as preflight_pbp.py. Run it before invoking the legacy signer.

```
#!/usr/bin/env python3
"""A conservative, bounded preflight for the pinned legacy ebootsign path."""
import os
import struct
import sys

path = sys.argv[1] if len(sys.argv) > 1 else "EBOOT.PBP"
with open(path, "rb") as f:
    total = os.fstat(f.fileno()).st_size
    header = f.read(40)
    if len(header) != 40 or header[:4] != b"\0PBP":
        raise SystemExit("Invalid PBP header")
    offsets = struct.unpack_from("<8I", header, 8)
    if offsets[0] != 40 or offsets[-1] > total:
        raise SystemExit("Invalid section bounds")
    if any(a > b for a, b in zip(offsets, offsets[1:])):
        raise SystemExit("Unordered section offsets")
    length = offsets[7] - offsets[6]
    if not 52 <= length <= 4 * 1024 * 1024:
        raise SystemExit("DATA.PSP must fit the 4 MiB input buffer")
    f.seek(offsets[6])
    data = f.read(length)
if len(data) != length:
    raise SystemExit("Short DATA.PSP read")
if data[:7] != b"\x7fELF\x01\x01\x01":
    raise SystemExit("Need ELF32 little-endian, version 1")
if struct.unpack_from("<HH", data, 16) != (0xFFA0, 8):
    raise SystemExit("Need relocatable PSP PRX for MIPS")
print("Preflight OK:", length, "bytes")
```

The gate first checks the PBP section bounds, then computes `offset[7] - offset[6]`. It rejects an oversized DATA.PSP **before reading that payload**. The payload read is bounded to 4 MiB and then checked for the expected ELF identity, PRX type and MIPS machine.

```
python3 preflight_pbp.py EBOOT.PBP
# Expected for this example: Preflight OK: 158450 bytes
```

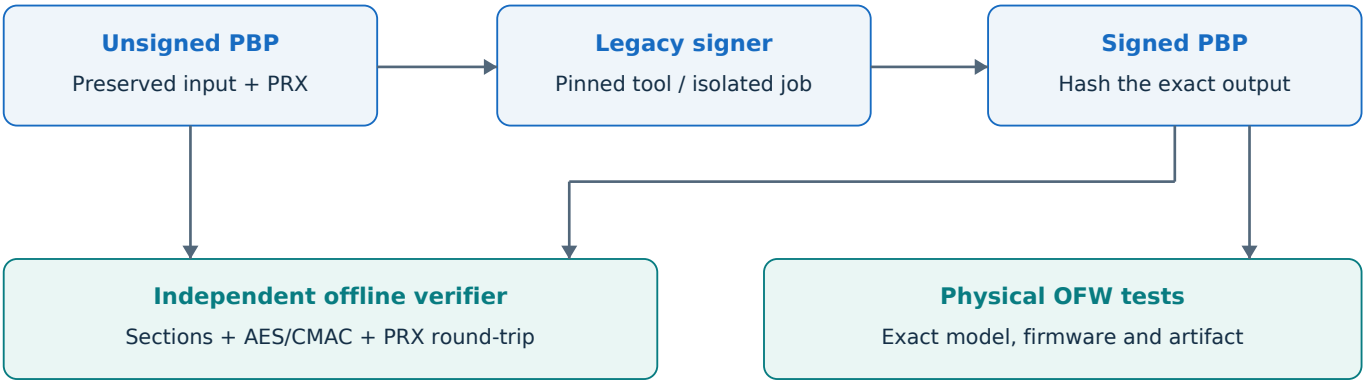
> **Deliberately narrow scope**
>
> This is a conservative gate for PBP files produced by the demonstrated build path. It expects the first section at byte 40. It is not a complete validator for arbitrary hostile ELF files: inspect section/program tables, module information, relocation targets and imports before using the legacy tool.

Do not replace these explicit checks with Python assert statements in a reusable gate: Python can remove asserts when optimization is enabled. For a file that can change concurrently, validate a private immutable input copy and sign that same copy.

Paranoid Cross-Validation

A layered method: independent observations, explicit expected differences, and negative controls.

Paranoid Cross-Validation means refusing to let a successful signer validate its own output by itself. Preserve the unsigned artifact, compute the expected transformation, examine the packed output, and verify the cryptography with an independent implementation. A physical OFW launch is a further gate.



A release claim requires the relevant evidence from both branches.

Gate	Evidence	What it cannot establish alone
Identity	SHA-256 for the input, tool and final output	Firmware authenticity or permissions.
Structure	PBP ranges, SFO values, ELF tables and imports	Cryptographic integrity or device acceptance.
Cryptography	Independent AES/CMAC and full PRX-prefix comparison	The full loader's rules or actual runtime behavior.
Hardware	Exact artifact exercised on named model / OFW	Untested models, firmware versions or app features.

State the expected difference explicitly: DATA.PSP changes from a raw PRX to an encrypted wrapper. Other PBP sections must match the preserved input. The decrypted program must match the input after the documented relocation adjustment, not merely start with ELF magic.

The verifier on pages 14-16 uses the separate Python cryptography library for AES and CMAC, instead of reusing the signer's C cryptographic routines. It still depends on the pinned format templates. That shared assumption is recorded rather than treated as independent proof of the complete firmware path.

Structural and permission checks

Read the correct layer, and compare everything that should remain unchanged.

Check	Required observation for this example
PBP header	Magic 00 50 42 50; version 0x00010000; eight ordered in-file offsets.
PBP resources	All seven non-DATA.PSP sections match the unsigned input byte for byte.
SFO	TITLE=SampleApp; CATEGORY=MG; BOOTABLE=1; MEMSIZE=0; PSP_SYSTEM_VER=6.60.
Unsigned ELF	ELF32 / little-endian / MIPS; e_type 0xFFA0; valid table and section ranges.
Module attributes	Inner .rodata.sceModuleInfo attribute is 0x0000.
Main thread	Source requests PSP_THREAD_ATTR_USER; inspect the linked parameter data as part of an audit.
Imports	Inventory library + NID + version/attributes; map to the intended public user stubs.
Relocations	Compute the expected changes using the pinned type-7 rule; this sample required zero changes.

The signed ~PSP template's outer attribute is `0x0200`, while SampleApp's inner module attribute is `0x0000`. These are fields in different structures. Read each according to its own format instead of treating the wrapper value as the application's source declaration [S2, S4].

For ELF tables, validate `offset + count * entry_size` before iterating. File-backed sections need in-file ranges. SHT_NOBITS describes memory without occupying that many bytes in the file. Program segment file sizes and memory sizes are also distinct.

> **Hashing has a precise purpose**
>
> SHA-256 proves that two copies match a known artifact identity when the expected digest is trusted. It does not turn an unsigned program into a signed one, validate a CMAC or establish that its imports are legal on OFW.

The compact offline verifier performs selected structural checks, section comparison and crypto round-trip. It does not implement an exhaustive ELF semantic audit, NID permission database or firmware module loader. Retain the manual/SDK import audit and physical test gate.

Inside the crypto cross-check

Reconstruct the pinned legacy layout, validate both CMACs, then compare the decrypted program.

The signed DATA.PSP begins with a 0x150-byte ~PSP header. This verifier checks it against the pinned template. It then reconstructs the KIRK-style input by prepending the template's 0x110-byte KIRK header to the encrypted body from the signed payload [S4].

Reconstructed raw range	Role
0x00..0x1F	Encrypted AES payload key and AES-CMAC key.
0x20..0x2F	Expected header CMAC.
0x30..0x3F	Expected body/data CMAC.
0x60..0x8F	Header fields authenticated by the header CMAC.
0x70 / 0x74	DATA size = 0x553300; offset = 0x80.
0x90 + 0x80 = 0x110	Start of the encrypted, aligned program body.

```
aligned = (data_size + 15) & ~15
end = 0x90 + data_offset + aligned

header_cmac = CMAC(cmac_key, raw[0x60:0x90])
body_cmac   = CMAC(cmac_key, raw[0x60:end])
plain       = AES_CBC_DECRYPT(aes_key, raw[0x110:end])
expected    = INPUT_PRX_WITH_DOCUMENTED_RELOCATION_FIXES
require(plain[:len(expected)] == expected)
```

The notation above describes the operation; the runnable implementation follows. AES-CBC uses a zero IV in this pinned path. The two keys are obtained by decrypting the first 32 reconstructed bytes with the upstream implementation's KIRK-1 constant. No secret from a user's PSP or console dump is needed for this offline recipe.

The signed payload has a fixed length of **5,583,952 bytes**, or `0x553450`. The decrypted prefix must equal the entire expected PRX. Do not require every trailing padding byte to be zero: the legacy CMAC correction modifies part of the padded body.

> **Proof boundary**
>
> These checks establish that the artifact matches this template, its specified CMACs and the expected program transformation. They do not independently validate every outer firmware header rule or prove that a physical OFW loader will accept the program.

Independent verifier: 1 / 3

Save the three consecutive code blocks on pages 14-16 into one file: cross_validate.py.

Prepare the pinned source directory used by the verifier. The header arrays and constant are read from that public checkout; literal key bytes are not embedded in this guide. The script checks the exact Git blob identities of its two source files.

```
mkdir -p tools
git clone https://github.com/pspdev/ebootsigner.git tools/ebootsigner
git -C tools/ebootsigner checkout --detach \
    17d6386f034ac922f540ca78200961761b23ecae
python3 -m venv .venv
. .venv/bin/activate
python -m pip install cryptography
```

```
#!/usr/bin/env python3
"""Offline checks for one pinned ebootsign format, not an OFW emulator."""
import hashlib
import hmac
import re
import struct
import sys
from pathlib import Path
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.cmac import CMAC

def need(ok, message):
    if not ok:
        raise ValueError(message)

def pbp(path):
    b = Path(path).read_bytes()
    need(len(b) >= 40 and b[:4] == b"\0PBP", "PBP header")
    need(struct.unpack_from("<I", b, 4)[0] == 0x10000, "PBP version")
    o = struct.unpack_from("<8I", b, 8) + (len(b),)
    need(o[0] == 40 and all(a <= z for a, z in zip(o, o[1:])),
         "PBP bounds")
    return [b[a:z] for a, z in zip(o, o[1:])]

def source(root, filename, blob):
    b = (root / filename).read_bytes()
    prefix = b"blob " + str(len(b)).encode() + b"\0"
    need(hashlib.sha1(prefix + b).hexdigest() == blob,
         "Pinned source changed: " + filename)
    return b.decode("ascii")

def array(text, name, size):
    pattern = r"\b" + name + r"\s*\[[^]]*\]\s*=\s*\{(.*?)\};"
    m = re.search(pattern, text, re.S)
    need(m is not None, "Missing C array: " + name)
    b = bytes(int(x, 16) for x in
              re.findall(r"0x([0-9a-fA-F]{1,2})\b", m[1]))
    need(len(b) == size, "C array length: " + name)
    return b
```

The format profile is intentionally strict. A different wrapper or a modified source template is rejected rather than guessed. SHA-1 here checks Git's blob identity for the pinned source files; distribution checksums for executable artifacts use SHA-256.

Independent verifier: 2 / 3

Continue the same file immediately after the previous page's code.

```
def expected_prx(data):
    b = bytearray(data)
    need(52 <= len(b) <= 4 * 1024 * 1024, "4 MiB input bound")
    need(b[:7] == b"\x7fELF\x01\x01\x01", "ELF identity")
    need(struct.unpack_from("<HH", b, 16) == (0xFFA0, 8),
         "PRX type / MIPS machine")
    off = struct.unpack_from("<I", b, 32)[0]
    step, count = struct.unpack_from("<HH", b, 46)
    need(step == 40 and off + step * count <= len(b),
         "ELF section table")
    changed = 0
    for i in range(count):
        s = struct.unpack_from("<10I", b, off + i * step)
        start, size = s[4:6]
        if s[1] != 8:  # SHT_NOBITS has no file payload.
            need(start + size <= len(b), "ELF section bounds")
        if s[1] == 0x700000A0:
            need(size % 8 == 0, "Relocation alignment")
            for p in range(start, start + size, 8):
                info = struct.unpack_from("<I", b, p + 4)[0]
                if info & 0xFF == 7:
                    struct.pack_into("<I", b, p + 4, info & 0xFFFFFF00)
                    changed += 1
    return bytes(b), changed

def decrypt(key, data):
    ctx = Cipher(algorithms.AES(key), modes.CBC(bytes(16))).decryptor()
    return ctx.update(data) + ctx.finalize()

def check_cmac(key, data, expected):
    ctx = CMAC(algorithms.AES(key))
    ctx.update(data)
    need(hmac.compare_digest(ctx.finalize(), expected), "CMAC mismatch")
```

`expected_prx` copies the unsigned input and reproduces only the documented relocation adjustment. It validates the section-table range and file-backed section ranges before iterating the relocation records. SHT_NOBITS is excluded from file-payload checks because its content is allocated in memory.

The actual SampleApp input had **zero** type-7 relocation records requiring a change. For another program, a nonzero count is not automatically a defect; it is a recorded expected transformation that must agree with the inspected signer path.

> **Independent primitives**
>
> The Python implementation delegates AES and CMAC to cryptography, not to the C routines linked into ebootsign. It uses explicit exceptions for failed requirements and a constant-time digest comparison for CMAC values. This reduces a common-source crypto implementation dependency, while the fixed layout remains a shared assumption.

The code is intended for the trusted, locally built artifacts in this workflow. It reads PBP files into memory and is not a general-purpose hostile-file sandbox. The bounded gate on page 10 must run before the legacy signer; a full ELF/import audit remains separate.

Independent verifier: 3 / 3

Finish the file with this block, then run it against the preserved input and new signed output.

```
def verify(unsigned, signed, root):
    u, s = pbp(unsigned), pbp(signed)
    need(all(u[i] == s[i] for i in range(8) if i != 6),
         "Non-DATA.PSP section changed")
    crypter = source(root, "main_crypter.h",
        "49c1db30e4ae5c4c54b1340718972d4e56d56805")
    keys = source(root, "internal_keys.h",
        "905c6c2f8c722c67261f5342ecb4cdd6f85fdaed")
    header = array(crypter, "pspHeader", 0x150)
    template = array(crypter, "kirkHeader", 0x110)
    master = array(keys, "kirk1_key", 16)
    need(len(s[6]) == 0x553450, "Signed payload size")
    need(s[6][:0x150] == header, "PSP wrapper template")
    raw = template + s[6][0x150:]
    need(struct.unpack_from("<I", raw, 0x60)[0] == 1, "KIRK mode")
    size, offset = struct.unpack_from("<II", raw, 0x70)
    need((size, offset) == (0x553300, 0x80), "KIRK layout")
    aligned = (size + 15) & ~15
    end = 0x90 + offset + aligned
    need(end == len(raw), "KIRK bounds")
    decoded = decrypt(master, raw[:32])
    check_cmac(decoded[16:], raw[0x60:0x90], raw[0x20:0x30])
    check_cmac(decoded[16:], raw[0x60:end], raw[0x30:0x40])
    plain = decrypt(decoded[:16], raw[0x90 + offset:end])
    expected, changed = expected_prx(u[6])
    need(plain[:len(expected)] == expected, "Decrypted PRX mismatch")
    return {"input_bytes": len(u[6]), "signed_bytes": len(s[6]),
            "relocations_changed": changed, "offline_checks": "PASS"}

if __name__ == "__main__":
    if len(sys.argv) != 4:
        raise SystemExit("Usage: cross_validate.py unsigned.pbp signed.pbp sources")
    print(verify(sys.argv[1], sys.argv[2], Path(sys.argv[3])))
```

```
python cross_validate.py EBOOT.PBP build/EBOOT-signed.PBP \
    tools/ebootsigner
```

Expected output for the measured example:

```
{'input_bytes': 158450, 'signed_bytes': 5583952,
 'relocations_changed': 0, 'offline_checks': 'PASS'}
```

Any mismatch raises an exception and returns failure. Keep the console output with the artifact hashes. A PASS is deliberately labeled **offline_checks**; it must not be renamed to 'OFW verified' without the physical tests on page 20.

Prove the checker can fail

Negative controls protect against a validator that accepts every file or checks the wrong object.

All the following damaged controls were run against copies of the measured sample. Each mutation was confined to a temporary file. The release candidate and unsigned reference remained intact.

Control	Expected rejection	Observed
Flip an encrypted-body byte	Body/data CMAC mismatch	Rejected
Flip a ~PSP header byte	Wrapper template mismatch	Rejected
Change a PARAM.SFO byte	Non-DATA.PSP section mismatch	Rejected
Truncate the signed PBP	Container bounds failure	Rejected
Supply a different original PRX	Decrypted PRX prefix mismatch	Rejected

The size gate was separately exercised at 4 MiB minus one byte, exactly 4 MiB, and 4 MiB plus one byte. It accepted the first two sizes and rejected the oversized input. Reversed section offsets, a truncated payload, a wrong ELF machine and a payload shorter than an ELF32 header were also rejected.

> **Do not confuse fixtures with programs**
>
> The boundary fixtures only carried the header fields needed by the basic gate. They were not complete runnable PRXs. Testing the gate's inclusive size boundary is distinct from validating an executable. No oversized fixture was passed into the unsafe legacy read path.

Repeatability control

The same unsigned input was signed again with the same installed binary in another clean job directory. The two signed outputs were byte-identical, and the independent verifier passed for both. This is an observed result for this input and tool, not a promise that all toolchains, metadata timestamps or signing methods are reproducible.

When adding a new check

Add one corresponding damaged control that should fail only that property. For example, a resource comparison needs a resource mutation; a program round-trip needs a mismatched original. A test that only prints ELF magic cannot validate the entire executable.

Measured SampleApp audit record

These values identify the actual artifacts created for this guide, not all possible builds.

Property	Measured result
Build environment	Ubuntu 24.04 x86_64; PSPDEV v20261001; psp-gcc 15.2.0
Unsigned EBOOT.PBP	158,806 bytes
Unsigned DATA.PSP	158,450 bytes; ELF32 / MIPS / PRX 0xFFA0
Signed EBOOT.PBP	5,584,308 bytes
Signed DATA.PSP	5,583,952 bytes; fixed ~PSP envelope
Module / SFO	Inner attribute 0x0000; MEMSIZE=0; PSP_SYSTEM_VER=6.60
Offline cross-validation	PASS: resources, template, two CMACs and decrypted PRX prefix
Relocations changed	0
Damaged controls	5 / 5 rejected; 7 size/format gate fixtures behaved as expected
Repeated signing	Same input + same binary: byte-identical output
Physical PSP / OFW	Not tested for this example

Unsigned EBOOT.PBP / SHA-256

```
f9a473323558989437d20cfe26bd7771
3c298aa006dd84df8287adf54a21fbaa
```

Signed EBOOT.PBP / SHA-256

```
f0fb983730a90bfee75a48a3b911e277
409c8466654151f251664dcd4674e4a2
```

Installed ebootsign binary / SHA-256

```
553b944f5f0df3a5549f8a1d7533cce0
dbdbd3e92f261a4c6774263b7e597c47
```

Each SHA-256 value is displayed on two lines; concatenate them without whitespace. A locally rebuilt artifact can have another digest. Record that new digest and repeat its checks instead of forcing a match to this reference build.

File size is not a RAM budget

The host signer's array, the fixed output wrapper and the PSP application occupy different layers.

Quantity	Value here	Meaning
Host input array	4 MiB	Raw PRX capacity in the legacy signer on the computer.
Signed DATA.PSP	5,583,952 bytes	Fixed wrapper/body in the stored EBOOT; not live application RAM.
Application heap request	2 MiB	PSP_HEAP_SIZE_KB(2048) in this program.
Main-thread stack	32 KiB	PSP_MAIN_THREAD_STACK_SIZE_KB(32).
ELF LOAD memory size	0x25804 = 153,604 bytes	Recorded segment memory size; excludes separate heap/runtime allocations.

The PSP-1000 has 32 MiB of physical main RAM; later PSP families have more physical RAM. An OFW user application does not automatically own all of it. Firmware reservations, loaded modules, allocations and loader behavior affect what is actually available. MEMSIZE=0 is a configuration choice, not a measured free-memory guarantee.

Plan for the smallest target

Budget the loaded code/data and BSS, requested heap, every thread stack, decoded assets, I/O buffers and subsystem allocations separately. Stream large files, bound queues, and fail cleanly on allocation errors. Avoid loading an entire asset library just because the card has space.

For an expanded application, measure free memory after initialization and at the feature's peak allocation point. Record allocation failures and sustained behavior on PSP-1000. Confirm module-loading and loader scratch-memory behavior on hardware rather than inferring it from the PBP's final disk size.

CPU and thread behavior

SampleApp does not change the clock speed. Its callback-aware delay yields CPU time instead of spinning in a busy loop. An application's own performance goals at 222 MHz need measurements; they are not established by signing, a User Mode flag or this text-only demonstration.

VFPU thread attributes, when needed by a different application, concern vector/FPU context handling. They are not a substitute for the USER attribute and do not by themselves grant kernel privileges [S2].

The physical OFW test gate

A passing offline audit becomes a firmware claim only when the exact artifact is tested on the stated hardware.

Copy the signed output as `PSP/GAME/SampleApp/EBOOT.PBP` on the mounted storage. Use the corresponding internal-storage GAME location for PSP Go when appropriate. Recompute SHA-256 after the copy to detect transfer errors.

```
PSP/
  GAME/
    SampleApp/
      EBOOT.PBP
```

Target family	OFW 6.60	OFW 6.61	Current example evidence
PSP-1000	Pending	Pending	No physical test
PSP-2000	Pending	Pending	No physical test
PSP-3000	Pending	Pending	No physical test
PSP Go	Pending	Pending	No physical test
PSP Street / E1000	Pending	Pending	No physical test

For each result, record the exact PSP model, firmware version, how the stock firmware state was established, artifact SHA-256, installation path and test date. A test on CFW can help diagnose runtime defects but does not establish stock OFW acceptance. Firmware version text alone may be insufficient to distinguish a modified environment.

Run a concrete checklist

Confirm XMB listing and title, cold launch after a restart, visible text, responsive HOME action, clean return to XMB, repeated launches, and suspend/resume behavior. For a larger application, exercise its real I/O, graphics, audio and worst-case memory paths. Record observed failures as carefully as successes.

> **Publish the scope you actually tested**
>
> 'Tested on PSP-3000 with OFW 6.61 using SHA-256 ...' is stronger evidence than 'works on all PSPs.' For this neutral example, all physical-device cells remain pending. Emulator behavior also cannot replace stock hardware evidence.

This launch workflow installs an application on storage; it requires no firmware flashing or console dump publication. A failed launch is a diagnostic result, not a reason to modify flash memory.

Troubleshooting by layer

Choose the next investigation from the failing gate, and preserve the failing artifact.

Symptom	First investigation
Signer rejects 'not a PRX'	Check DATA.PSP identity and e_type; confirm BUILD_PRX=1 and that you signed the unsigned PBP.
Preflight rejects the input	Read the exact message. Inspect PBP ranges, DATA.PSP size and ELF identity before invoking the signer.
No output, partial output or inconsistent jobs	Use separate clean job directories; review tool logs and hashes; inspect the fixed tmp_sgn behavior.
Wrapper/template mismatch	Check signer version/format and pinned source files; this verifier intentionally accepts one profile.
CMAC fails	Check exact input/output copies, ciphertext integrity and reconstructed layout. Do not distribute that candidate.
Decrypted PRX differs	Confirm the preserved unsigned input, relocation rule and that the correct candidate was checked.
App absent or shown as corrupted	Check the GAME subdirectory, filename, PBP resources, SFO and storage copy.
Launch error despite offline PASS	Investigate firmware loader rules, imports/NIDs, module availability, metadata and permissions on that exact OFW.
Starts, then freezes or HOME stalls	Investigate runtime allocations, thread behavior and callback servicing; signing cannot repair program logic.

Record a launch error code verbatim and keep the model, firmware and candidate digest beside it. Avoid assigning one universal cause to an error code without confirming its context. Preserve a known-good build so a new toolchain or feature can be compared against it.

Re-signing is not a permission repair

A user module that imports a private CFW service remains dependent on that service after encryption. Replace the unsupported call path, rebuild the PRX, inspect its imports and sign the new candidate. Changing only a wrapper field can introduce a different mismatch without fixing the program.

If changes to the tool itself are necessary, move bounds checks before reads, verify I/O return values, validate ELF tables before relocation processing, use unique temporary directories and publish the changed source revision. Re-run negative controls and physical tests for that tool's output.

A release others can inspect

Make download, installation and compatibility evidence easy to separate and reproduce.

Include	Purpose
PSP/GAME/SampleApp/EBOOT.PBP	The signed candidate under the conventional installed filename.
SHA256SUMS.txt	Artifact identity and a way to verify the copied file.
README.md	Installation, controls, feature scope and exact compatibility status.
Source + Makefile	Rebuild and dependency review; preserve the unsigned input in the audit workspace.
VALIDATION.md	Tools, revisions, hashes, import audit, offline checks, negative controls and hardware results.
LICENSE / notices	Your source license plus relevant third-party license and attribution information.

```
mkdir -p release/PSP/GAME/SampleApp
cp build/EBOOT-signed.PBP release/PSP/GAME/SampleApp/EBOOT.PBP
(
    cd release
    sha256sum PSP/GAME/SampleApp/EBOOT.PBP > SHA256SUMS.txt
    zip -r ../SampleApp-OFW-candidate.zip PSP SHA256SUMS.txt
)
```

Add the documentation and license files to the release staging directory before archiving if you want them in the download. Keep development intermediates, private configuration and device dumps out of the distributable. The signed application does not require someone else's flash files.

Accurate example wording

SampleApp is a native PSP User Mode demonstration. Its unsigned and signed containers passed the documented independent offline checks. Five damaged controls were rejected. Physical OFW 6.60 / 6.61 compatibility has not yet been tested for this artifact.

When hardware tests are completed, replace only the corresponding status fields with the exact model, firmware and hash. If the executable changes, those old physical results describe the old artifact; repeat the relevant tests rather than carrying the result forward automatically.

The original SampleApp and Python example code printed in this guide may be reused under the MIT license. External tools and source files retain their own licenses. This document explains a reviewed build path; it is not a Sony certification or an all-model compatibility guarantee.

Sources and provenance

Primary implementation sources and developer documentation; inspected 11 October 2026.

S1 PSPDEV and its native examples

https://pspdev.github.io/

Installation and project entry point. Basic programs show public User Mode module declarations and PSP build patterns.

https://pspdev.github.io/basic_programs.html

S2 PSPSDK module and thread definitions

https://pspdev.github.io/pspsdk/pspmoduleinfo_8h_source.html

Module attributes and PSP_MODULE_INFO. Thread documentation below defines USER / VFPU attributes and callback-aware thread functions.

https://pspdev.github.io/pspsdk/pspthreadman_8h_source.html

S3 SDK user import stubs

https://github.com/pspdev/pspsdk/blob/e2b2b313d4175823a263c2c6cf175311e824eb77/src/user/Kernel_Library.S

Primary NID-to-symbol definitions for the discussed Kernel_Library user stubs. Review each imported library's corresponding source and your installed archive.

S4 Pinned legacy signer

https://github.com/pspdev/ebootsigner/tree/17d6386f034ac922f540ca78200961761b23ecae

Files inspected: ebootsign.c, main_crypter.h, kirk_engine.c/.h, internal_keys.h, fix-realocations.c, pack-pbp.c, unpack-pbp.c and Makefile. These ground the buffer, template, AES/CMAC and relocation discussion.

S5 SDK packaging rules

https://github.com/pspdev/pspsdk/blob/e2b2b313d4175823a263c2c6cf175311e824eb77/src/base/build.mak

BUILD_PRX, PSP_FW_VERSION, PSP_LARGE_MEMORY, SFOFLAGS and PBP build behavior. tools/mksfoex.c supplies the associated SFO generation implementation.

Separate the build from the references

The measured build used **PSPDEV v20261001, psp-gcc 15.2.0, on Ubuntu 24.04 x86_64**. The SDK reference links are pinned to a later source snapshot (8 October 2026); they are not presented as the exact source revision of every binary in the 1 October toolchain bundle.

The installed signer binary is identified by its SHA-256 on page 18. The verifier checks pinned template source blobs and observed output properties. Reproducing the binary itself requires recording the build compiler, flags and full dependency provenance. Preserve your own audit record when those inputs change.